

Clasificación de Enfermedades en Hojas de Cultivos mediante Redes Neuronales Convolucionales y Transfer Learning



Proyecto – Entrega 2

Fundamentos de Deep Learning

Daniel Alejandro Yepes Mesa

Universidad de Antioquia
Facultad de Ingeniería
Medellín
2026

1. Resumen

En este proyecto aplique CNNs y Transfer Learning para clasificar enfermedades en hojas de plantas usando el dataset PlantVillage (54,305 imágenes, 38 clases). Compare cuatro modelos: una CNN desde cero como baseline y tres arquitecturas pre entrenadas (VGG16, InceptionV3 y ResNet50). El mejor resultado lo obtuvo ResNet50 con 97.75% de accuracy en test, frente al 62.77% del baseline. Al final use GradCAM para analizar que zonas de la imagen activa el modelo al clasificar.

2. El problema

El problema que trabaje es clasificación automática de enfermedades en plantas a partir de fotos de hojas. Dado que muchos cultivos se pierden cada año por enfermedades que se detectan tarde, la idea es que un modelo pueda identificar la enfermedad a partir de una foto, sin necesidad de un experto agrónomo en el campo.

Use el dataset PlantVillage, que tiene 54,305 imágenes JPEG de 256x256 pixeles de hojas de 14 cultivos distintos. Las imágenes están etiquetadas en 38 clases entre estados saludables y con distintas enfermedades. El split que usé fue 70% entrenamiento, 15% validación y 15% test. El reto principal no es solo que son 38 clases, sino que el dataset esta bastante desbalanceado. Hay clases con mas de 5,000 imágenes y otras con menos de 300. Si no se maneja eso, el modelo aprende a ignorar las clases minoritarias porque el error en esas clases tiene poco peso en la perdida total. Ese fue uno de los primeros problemas que tuve que resolver.

3. Estructura de los notebooks

Notebook	Contenido
01 - Exploración de datos	Análisis del dataset, distribución de clases, visualización de muestras y estadísticas por canal RGB.
02 - Preprocesado	Pipeline de augmentation y normalización, cálculo de pesos de clase para manejar el desbalance.
03 - Modelo baseline CNN	CNN desde cero con imágenes de 64x64 como punto de comparación base.
04 - Transfer Learning VGG16	Feature extraction y fine-tuning con VGG16 preentrenada en ImageNet.
05 - Transfer Learning ResNet50	Feature extraction y fine-tuning con ResNet50 preentrenada en ImageNet.

06 - Transfer Learning InceptionV3	Feature extraction y fine-tuning con InceptionV3 preentrenada en ImageNet.
07 - Comparación y GradCAM	Comparativa de los cuatro modelos, visualización con GradCAM y análisis de errores.

Todos los notebooks se pueden ejecutar desde cero en cualquier cuenta de Google Colab. El dataset se descarga automáticamente con kagglehub sin necesidad de credenciales y los modelos pre entrenados los descarga Keras desde sus repositorios oficiales. No se necesita configuración adicional.

4. Exploración de datos

Lo primero que hice antes de entrenar cualquier cosa fue entender bien el dataset. Cargue PlantVillage con kagglehub y conté las imágenes por clase en el split de entrenamiento.

Lo que encontré confirmó el problema del desbalance: la clase con más imágenes tiene 5,357 ejemplos y la que menos tiene tiene 275. Eso es una diferencia de casi 20 veces entre extremos. Con ese nivel de desbalance un modelo sin ninguna compensación va a tender a predecir siempre las clases mayoritarias porque el error en las clases chicas es casi insignificante.

También calcule la media y desviación estándar por canal RGB sobre una muestra de 800 imágenes. Eso me sirvió para confirmar que la normalización a $[0, 1]$ era suficiente y que no necesitaba normalizar por canal como se hace a veces con ImageNet.

5. Preprocesado y pipeline de datos

El pipeline de datos lo arme en el notebook 02 y lo reutilizo en todos los entrenamientos. Use 224x224 pixeles porque es el tamaño estándar que esperan VGG16 y ResNet50. Para InceptionV3 use 160x160 para reducir el cómputo, cosa que la arquitectura permite porque quite la cabeza original.

Las decisiones de augmentation que tomé fueron:

Transformacion	Por que la use
RandomFlip horizontal	Las hojas no tienen una orientación fija, tiene sentido verlas en espejo.
RandomRotation (+-10%)	Las fotos de campo no siempre estan derechas.
RandomZoom (+-10%)	Las distancias de captura varían, quería que el modelo fuera robusto a eso.
RandomBrightness y RandomContrast	La iluminación cambia mucho entre fotos tomadas a distintas horas o condiciones.

Para el desbalance de clases use `class_weight`. La idea es que las clases con pocos ejemplos reciban un peso mayor en la función de pérdida, de modo que un error en esas clases cueste más que un error en una clase con miles de imágenes. Los pesos los calcule sobre el split de

entrenamiento y los guarde en un JSON para utilizarlos en todos los notebooks de entrenamiento.

Una decisión que tomé fue poner el augmentation dentro del modelo como capas de Keras, no aplicarlo antes en el pipeline. Así el augmentation corre en GPU y no se convierte en un cuello de botella en la carga de datos.

6. Iteraciones de modelado

Hice cuatro iteraciones en total, partiendo de una CNN desde cero y escalando hasta transfer learning con tres arquitecturas distintas. En todas use EarlyStopping y ModelCheckpoint para guardar el mejor modelo según la pérdida de validación.

6.1) CNN desde cero (baseline)

La primera iteración fue una CNN construida desde cero para tener un punto de partida claro. Use imágenes de 64x64 porque entrenar con 224x224 desde cero en Colab gratis hubiera sido muy lento y tampoco esperaba que fuera el mejor modelo.

La arquitectura tiene cuatro bloques de Conv2D seguidos de BatchNormalization, activación ReLU y MaxPooling2D. Al final uso GlobalAveragePooling2D para reducir dimensiones antes de las capas densas, con Dropout de 0.4 para regularizar. El augmentation y la normalización van dentro del modelo como capas de Keras.

El modelo convergió en 23 épocas con EarlyStopping y llegó a **62.77% de accuracy en test**. Es un resultado razonable para una red desde cero con imágenes pequeñas y 38 clases, pero deja claro que hay mucho margen de mejora.

6.2) Transfer Learning: flujo general

Para los tres modelos de transfer learning use el mismo flujo de dos fases. La lógica es la misma en todos los casos:

Fase 1 - Feature extraction: cargo la base pre entrenada en ImageNet sin la cabeza original y la congelo completamente. Le agregé una cabeza nueva: GlobalAveragePooling2D, una capa densa con Dropout y la capa de salida con 38 unidades y activación softmax. Entreno solo la cabeza nueva con learning rate alto ($1e-3$). Esto converge rápido porque la base no cambia y los pesos de ImageNet ya son buenos.

Fase 2 - Fine-tuning: descongelo las últimas capas de la base y sigo entrenando con un learning rate mucho más bajo ($1e-5$), para ajustar esos filtros al problema de plantas sin destruir lo que ya aprendieron. Cual bloque descongelo depende de la arquitectura.

6.3) VGG16

VGG16 fue el primer modelo de transfer learning que probe. En la fase 2 descongelo las últimas 4 capas del bloque 5 (block5_conv1 hasta block5_pool). La red es grande (138M parámetros totales) pero entrenables en fase 2 son solo unos 7.2M.

Entrenamiento: 10 epocas en fase 1 y 10 en fase 2. Resultado en test: **66.70%**. Una mejora sobre el baseline, pero mas modesta de lo que esperaba.

6.4) ResNet50

ResNet50 fue la sorpresa del proyecto. La diferencia clave con VGG16 es que ResNet usa conexiones residuales: cada bloque suma su entrada a su propia salida, lo que permite entrenar redes mucho mas profundas sin que el gradiente se desvanezca. Eso le da capacidad para aprender patrones mas finos.

En la fase 2 descongele el bloque conv5, que es el de más alto nivel semántico de la red. Entrenamiento: 8 epocas en fase 1 y 10 en fase 2.

Resultado en test: **97.75%**. Ese salto fue inesperado para mi, casi 31 puntos porcentuales por encima de VGG16.

6.5) InceptionV3

Con InceptionV3 use 160x160 en lugar de los 299x299 originales para reducir el computo. Como quite la cabeza original, la arquitectura acepta esa resolución sin problema. En la fase 2 descongele los bloques mixed9 y mixed10, que son los de mas alto nivel.

Entrenamiento: 10 epocas en fase 1 y 10 en fase 2. Resultado en test: **71.76%**. Mejor que VGG16, pero bastante por debajo de ResNet50. El hecho de haber bajado la resolución probablemente afecto algo el resultado.

7. Comparación de resultados

Modelo	Test Accuracy	Epocas totales	Resolucion
CNN baseline	62.77%	23	64x64
VGG16	66.70%	20 (10+10)	224x224
InceptionV3	71.76%	20 (10+10)	160x160
ResNet50	97.75%	18 (8+10)	224x224

El resultado de ResNet50 fue el que mas me sorprendió. Esperaba mejoras sobre el baseline, pero no una diferencia tan grande entre ResNet50 y las otras dos. Creo que la combinación de las conexiones residuales con el fine-tuning del bloque conv5 le permite capturar patrones de textura y color muy específicos de cada enfermedad, que VGG16 e InceptionV3 no logran capturar igual de bien.

También puede influir que VGG16 es una red mas antigua y menos eficiente por diseño, y que InceptionV3 entreno con imagenes mas chicas (160x160) lo que le quita algo de detalle.

7.1) GradCAM

En el notebook 07 implementa GradCAM para visualizar qué regiones de la imagen activa el modelo al hacer una predicción. La idea es calcular los gradientes de la última capa convolucional respecto a la clase predicha y usarlos como pesos para armar un mapa de calor sobre la imagen.

Lo que encontré es que en las predicciones correctas el modelo enfoca las zonas con lesiones, manchas o decoloraciones en la hoja. En los errores, muchas veces activa zonas de fondo o el borde de la hoja, que no tienen información útil para distinguir enfermedades.

Al analizar los errores por clase, los fallos más frecuentes son entre enfermedades visualmente similares del mismo cultivo. Eso tiene sentido: si dos enfermedades producen manchas parecidas en las hojas de la misma planta, la distinción es difícil incluso para un humano sin entrenamiento.

8. Conclusiones

La conclusión mas clara del proyecto es que transfer learning supera ampliamente a una CNN desde cero para este tipo de problema, y que no todas las arquitecturas pre entrenadas dan el mismo resultado. ResNet50 fue significativamente mejor que VGG16 e InceptionV3 en las mismas condiciones de entrenamiento.

El manejo del desbalance de clases con pesos fue una decision que creo que vale la pena. Sin eso el modelo probablemente hubiera ignorado las clases con menos de 300 imágenes y el accuracy en esas clases especificas hubiera sido muy malo aunque el número global pareciera bueno.

Si tuviera más recursos y tiempo, probaría algunas cosas: primero, afinar más hiperparametros como el learning rate del fine-tuning o cuantas capas descongelar. Segundo, probar con las imágenes originales de 256x256 en InceptionV3 para ver si la reducción a 160x160 afectó mucho el resultado. Tercero, explorar técnicas de aumento de datos más agresivas para las clases minoritarias como sobremuestreo o generación sintética.

En general el proyecto me ayudó a entender de forma práctica porque transfer learning es la estrategia por defecto en clasificación de imágenes cuando se tiene un dataset de tamaño medio: los pesos de ImageNet ya capturan estructuras visuales útiles (bordes, texturas, formas) que son relevantes para casi cualquier tarea de visión, y el fine-tuning permite ajustarlas al problema específico sin necesitar millones de imágenes propias.

5. Referencias

- Mohanty, S.P., Hughes, D.P. & Salathe, M. (2016). Using Deep Learning for Image-Based Plant Disease Detection. *Frontiers in Plant Science*, 7, 1419.
- Too, E.C. et al. (2019). A comparative study of fine-tuning deep learning models for plant disease identification. *Computers and Electronics in Agriculture*, 161, 272-279.
- Selvaraju, R.R. et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *ICCV 2017*.

- He, K. et al. (2016). Deep Residual Learning for Image Recognition. CVPR 2016.
- Abdallahi, A. (2019). PlantVillage Dataset. Kaggle.
<https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset>